Dwield Passkey Server API Specification

Production Integration & REST Endpoint Reference — v0.1.0

Target Environment: Production / Staging	**Auth Protocol:** WebAuthn (FIDO2 Level 2/3)
Tenant Auth: SHA-256 API Key Header	**Identity Auth:** Customer Signed JWT / HMAC
Base URL: https://passkeys.dwield.com/v1	**Service Version:** 0.1.0 (Phases 1–6 Implemented)

1. System Architecture & Core Security Principles

The **Dwield Passkey Server** is a high-performance, multi-tenant WebAuthn authentication and adaptive step-up service. It acts as an independent cryptographic verification engine for customer applications. Dwield does not directly calculate user risk scores nor does it act as a primary identity provider; instead, customer backends maintain identity ownership and final access control.

- **Multi-Tenant API Isolation:** Customers are isolated into tenants identified by SHA-256 hashed API keys.
- **Decoupled Identity Context:** End-user identity is established via customer-signed JWTs or HMAC identity tokens.
- **Atomic Single-Use Challenges:** WebAuthn challenges and step-up tokens enforce strict single-use redemption with automatic MongoDB TTL cleanup.
- **Adaptive Step-Up Binding:** Risk assessments requiring STEP_UP are cryptographically bound to specific users, tenants, and sessions.
- **Zero Secret Exposure:** Public keys (COSE format) are stored server-side. Private keys never leave user devices.

2. Request Headers & Authentication Model

All API endpoints require tenant authentication via an API Key. Endpoints performing user-level ceremonies also require a validated User Identity Context Token.

Header Name	Type	Required	Description & Specification
`X-API-Key`	String	Yes	Dwield-issued API Key (e.g. dw_live_pub_...). Validated via SHA-256 lookup.
`X-Dwield-User-Token`	String	Conditional	Customer-issued user context token. Accepts JWT (sub, tenantId, exp) or HMAC format.
`Content-Type`	String	Yes	Must be application/json for all POST/DELETE requests.

3. Complete Endpoint Reference

[POST] `/v1/passkeys/registration/options` — Generate Passkey Registration Options

Initiates a WebAuthn passkey registration ceremony. Generates a cryptographically random challenge, populates excluded credentials, and returns standard FIDO2 options.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`{ "userName": "alex_dev", "userDisplayName": "Alex Dev" }`
Response (200 OK)	`{ "status": "success", "ceremonyId": "cer_reg_9a8f...", "options": { "rp": { "name": "Acme", "id": "acme.com" }, "challenge": "d34a...", "userID": "..." } }`
Possible Errors	401 UNAUTHORIZED (Missing/Invalid key or token), 403 FORBIDDEN (Tenant mismatch/suspended).

[POST] `/v1/passkeys/registration/verify` — Verify & Save Registered Passkey

Verifies the authenticator's WebAuthn registration response. Validates challenge, origin, RP ID, and user verification. Saves the COSE public key in MongoDB.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`{ "ceremonyId": "cer_reg_9a8f...", "response": { "id": "cred_123", "rawId": "...", "response": { "attestationObject": "...", "clientDataJSON": "..." }, "type": "public-key" }, "friendlyName": "MacBook TouchID" }`
Response (200 OK)	`{ "status": "success", "verified": true, "credentialId": "cred_123", "friendlyName": "MacBook TouchID", "createdAt": "2026-09-30T09:00:00Z" }`
Possible Errors	400 VALIDATION_ERROR (Invalid assertion signature/expired challenge), 401 UNAUTHORIZED.

[POST] `/v1/passkeys/authentication/options` — Generate Passkey Authentication Options

Generates WebAuthn authentication options for active credentials belonging to the user. Supports optional binding to a step-up session and risk assessment.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`{ "assessmentId": "asm_risk_771", "stepUpSessionId": "step_sess_441" }`
Response (200 OK)	`{ "status": "success", "ceremonyId": "cer_auth_811...", "options": { "challenge": "k92a...", "rpId": "acme.com", "allowCredentials": [{ "id": "cred_123", "type": "public-key" }] } }`
Possible Errors	404 NOT_FOUND (No active passkeys), 400 VALIDATION_ERROR (Invalid step-up session/mismatched assessment).

[POST] `/v1/passkeys/authentication/verify` — Verify WebAuthn Authentication Assertion

Verifies the browser's assertion response against the stored COSE public key. Increments signature counter, updates last used timestamp, and issues a signed Step-Up Token.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`{ "ceremonyId": "cer_auth_811...", "response": { "id": "cred_123", "rawId": "...", "response": { "authenticatorData": "...", "clientDataJSON": "...", "signature": "..." }, "type": "public-key" } }`
Response (200 OK)	`{ "status": "success", "verified": true, "tenantId": "tenant_acme", "userId": "usr_1001", "credentialId": "cred_123", "stepUpToken": "eyJqd..." }`
Possible Errors	401 UNAUTHORIZED (Passkey revoked or signature failure), 400 VALIDATION_ERROR (Challenge expired/replayed).

[GET] `/v1/passkeys` — List Active User Passkeys

Retrieves metadata for all active passkeys registered under the authenticated user identity for the current tenant.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`N/A (Query Parameters optional)`
Response (200 OK)	`{ "status": "success", "count": 1, "credentials": [{ "credentialId": "cred_123", "friendlyName": "MacBook TouchID", "deviceType": "singleDevice", "backedUp": false, "lastUsedAt": "2026-09-30T09:15:00Z", "createdAt": "2026-09-30T09:00:00Z" }] }`
Possible Errors	401 UNAUTHORIZED.

[DELETE] `/v1/passkeys/:credentialId` — Revoke Passkey Credential

Sets the credential status to revoked in MongoDB. Revoked passkeys immediately fail future authentication option generation and verification.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`N/A (credentialId passed as URL parameter)`
Response (200 OK)	`{ "status": "success", "message": "Passkey credential successfully revoked", "credentialId": "cred_123" }`
Possible Errors	403 FORBIDDEN (User does not own credential), 404 NOT_FOUND (Credential ID not found).

[POST] `/v1/step-up/initiate` — Initiate Adaptive Step-Up Session

Creates a short-lived Step-Up Session (status PENDING). Validates risk assessment with the Risk Engine and rejects DENY decisions.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key, X-Dwield-User-Token
Request Body	`{ "assessmentId": "asm_risk_771" }`
Response (201 Created)	`{ "status": "success", "session": { "stepUpSessionId": "step_sess_441", "tenantId": "tenant_acme", "userId": "usr_1001", "assessmentId": "asm_risk_771", "status": "PENDING", "expiresAt": "2026-09-30T09:20:00Z" } }`
Possible Errors	403 FORBIDDEN (Step-up forbidden for DENY risk decisions), 401 UNAUTHORIZED.

[POST] `/v1/step-up/verify-token` — Verify Signed Step-Up Token (Backend-to-Backend)

Called by the Customer Backend to verify the signed HMAC Step-Up Token. Atomically consumes token to prevent replay attacks and marks Step-Up Session as CONSUMED.

Field / Section	Specification & Sample Details
Headers Required	X-API-Key
Request Body	`{ "token": "eyJqd...", "expectedUserId": "usr_1001", "expectedAssessmentId": "asm_risk_771", "expectedStepUpSessionId": "step_sess_441" }`
Response (200 OK)	`{ "status": "success", "valid": true, "payload": { "jti": "stk_911...", "iss": "dwield-passkey-server", "sub": "usr_1001", "tenantId": "tenant_acme", "credentialId": "cred_123", "assessmentId": "asm_risk_771", "stepUpSessionId": "step_sess_441", "exp": 1790717340 } }`
Possible Errors	409 CONFLICT (Replay attempt: token already consumed), 401 UNAUTHORIZED (Signature invalid/mismatched identity).

[GET] `/health/ready` — Readiness & DB Health Probe

Used by load balancers and Kubernetes readiness probes to verify database connectivity.

Field / Section	Specification & Sample Details
Headers Required	None
Request Body	`N/A`
Response (200 OK)	`{ "status": "ready", "database": "connected", "timestamp": "2026-09-30T09:25:00Z" }`
Possible Errors	503 SERVICE_UNAVAILABLE (Database disconnected).

4. Standard Error Handling & Status Codes

All error responses adhere to a uniform JSON error structure across all endpoints.

```
{
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid user context token signature",
    "details": []
  }
}
```

HTTP Status	Error Code	Description & Trigger Condition
400 Bad Request	`VALIDATION_ERROR / BAD_REQUEST`	Zod schema validation failure, malformed JSON, or expired WebAuthn challenge.
401 Unauthorized	`UNAUTHORIZED`	Missing/invalid API key, missing/invalid user token, signature check failed, or revoked credential.
403 Forbidden	`FORBIDDEN`	Tenant mismatch between API key and user token, or step-up initiation for DENY decision.
404 Not Found	`NOT_FOUND`	No active credentials found for user, or requested endpoint path does not exist.
409 Conflict	`CONFLICT`	Replay attempt detected when redeeming single-use Step-Up Verification Token.
503 Service Unavailable	`SERVICE_UNAVAILABLE`	Readiness probe failure due to MongoDB disconnect.